

App de gestión de pedidos de tragos en boliches

Diseño funcional

1. Problema

En los boliches se generan largas filas para comprar y pedir tragos:

- **Modelo A:** fila para pagar en caja (te dan un ticket) + fila en barra para que te lo preparen.
- **Modelo B:** fila única en barra, donde cobran y preparan al mismo tiempo.

Ambos modelos generan demoras y pérdida de tiempo para el cliente.

2. Solución general

El cliente pide y paga el trago desde su celular, usando una **PWA** (web app instalable, sin pasar por ninguna store), **desde donde esté** (no hace falta estar parado en la barra para pedir) — y se acerca a retirarlo recién cuando está listo, evitando esperar parado mientras se prepara. El retiro es **asíncrono**: el cliente pide, sigue en lo suyo, y va hacia la barra cuando le avisan que está listo.

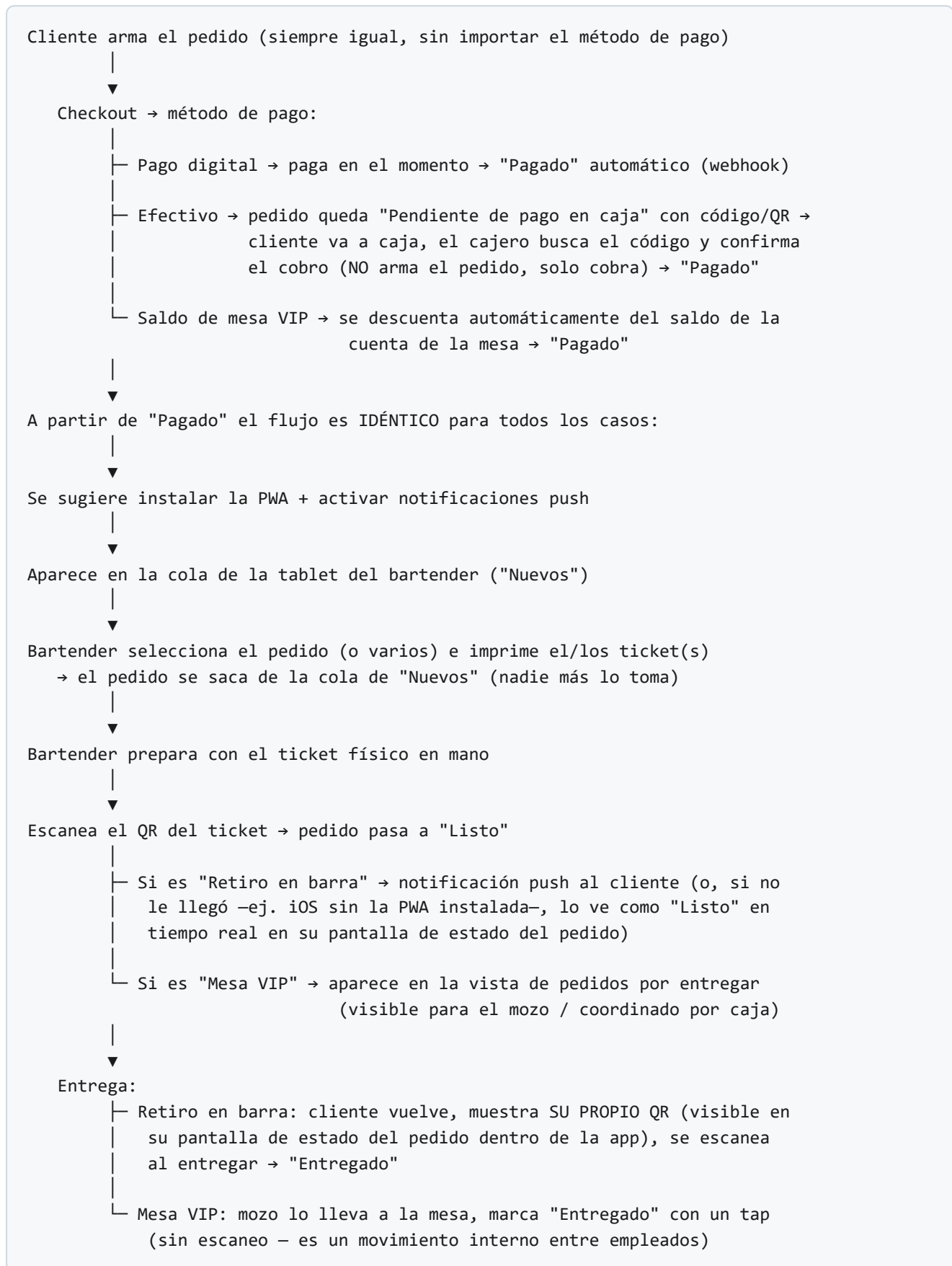
El aviso de "pedido listo" se dispara como **notificación push desde la propia PWA**. Justo después de confirmar el pago se le sugiere al cliente **instalar la PWA** (agregar a inicio) y aceptar el permiso de notificaciones — es el momento de mayor motivación (ya pagó, quiere que le avisen) y es un paso liviano de un par de taps, no de bajar una app de una store. Como red de respaldo para cuando la notificación no llega (típicamente **iOS sin la PWA instalada**, o permiso denegado), el cliente siempre puede volver a la pantalla de estado de su pedido, que se actualiza **en tiempo real** dentro de la misma app. Ver detalle en la sección 9.

3. Actores

- **Cliente:** arma y paga el pedido, lo retira o lo recibe en su mesa.
- **Cajero:** cobra pedidos con pago en efectivo; también cumple rol de coordinador de mozos en el sector VIP.
- **Bartender:** prepara los pedidos, los marca como listos.
- **Mozo:** entrega pedidos en mesas del sector VIP.
- **Admin/Encargado:** gestiona menú, mesas, usuarios, métricas (a definir en detalle más adelante).

4. Flujo general unificado del pedido

Los 3 casos de uso originales (efectivo, pago digital, VIP) convergen en **un solo flujo**, no en sistemas separados. La diferencia entre ellos está solo en *cómo se paga y cómo se entrega*, no en cómo se arma el pedido.



5. Estados del pedido (a nivel de pedido completo, no por ítem)

1. **Carrito** (armando el pedido)
2. **Pendiente de pago** (solo en caso efectivo, mientras espera pasar por caja)
3. **Pagado / Confirmado**
4. **En cola** (esperando ser tomado por un bartender)
5. **En preparación** (implícito: se imprimió, un bartender lo está haciendo)
6. **Listo** (bartender escaneó el QR del ticket)
7. **Entregado** (cerrado)
8. **Cancelado** (caso de timeout u otro motivo — pendiente de definir reglas exactas)

6. Caso 1 — Pago en efectivo

- El cliente **arma el pedido en la app igual que en los demás casos** (no es un flujo aparte).
- Al elegir "efectivo" como método de pago, el pedido queda con estado "Pendiente de pago en caja" y se genera un código/QR corto.
- El cliente se acerca a caja, el cajero **busca el pedido por ese código** (no lo tipea de cero) y confirma el cobro.
- Esto reduce drásticamente el tiempo en caja respecto al modelo actual, porque el cajero no arma pedidos, solo cobra.

7. Caso 2 — Pago digital

- Cliente arma el pedido y paga en el momento (tarjeta, Mercado Pago, etc.) desde la app/web.
- El webhook de la pasarela de pago confirma automáticamente el estado "Pagado".
- Sigue el flujo general de ahí en adelante.

8. Caso 3 — Sector VIP con mesas

8.1 Modelo de cuenta

- La mesa VIP se vende como un **paquete con presupuesto/saldo de consumo** (no se paga cada trago individualmente).
- Cada pedido hecho desde la mesa **descuenta automáticamente del saldo de esa cuenta**, sin elegir método de pago.
- El cliente ve su **saldo disponible** en la pantalla de menú al pedir desde una mesa VIP.
- Pendiente de definir: qué pasa cuando el saldo no alcanza para el pedido (bloquear, cubrir la diferencia con otro medio de pago, o permitir saldo negativo a saldar después). Se inclinó por

cubrir la diferencia con otro medio de pago como la opción más fluida, pero no quedó cerrado del todo.

8.2 Acceso a la cuenta / verificación

- **No es un QR público pegado en la mesa** (cualquiera que pase cerca podría gastar el saldo ajeno).
- Es un **único código/QR por mesa**, entregado **en persona por el mozo** a quien compró el paquete VIP (a mano o por WhatsApp) — no queda expuesto públicamente.
- La verificación de "quién es VIP" no la hace el sistema: la hace el control físico de acceso al sector VIP que el boliche ya tiene (portero, hostess, reserva). El sistema solo identifica la mesa/cuenta, no valida identidad de personas.
- No hace falta modelar "apertura/cierre de sesión de mesa" — cada pedido es independiente y se paga (descuenta) en el momento.

8.3 Entrega en mesa

- El pedido de mesa sigue el mismo ciclo que cualquier otro hasta "Listo" (bartender lo prepara e imprime igual que los demás, con la etiqueta "Mesa X" bien visible en la tablet en vez de "Retiro en barra").
- Al llegar a "Listo", en vez de notificar al cliente, aparece disponible para el mozo.
- El **reparto de mesas entre mozos** (zonas fijas vs. cola compartida) depende de cada boliche y queda **sin definir por ahora** — el diseño no obliga a resolverlo de antemano.
- El **cajero cumple un rol de coordinador**: como con el flujo unificado ya no arma pedidos ni gestiona una fila larga, tiene disponibilidad para ver qué mesas están "Listas" sin mozo asignado y coordinar a mano quién las lleva (de palabra, por radio, etc.) — el sistema solo expone la información, no fuerza un algoritmo de asignación.
- El mozo usa **su propio celular** (no una tablet dedicada), accediendo a una web app con un **login simple** (usuario/PIN) que lo identifica como empleado y le muestra la vista de pedidos de mesa listos para entregar.
- Marca "Entregado" con un tap simple, sin escaneo — es un movimiento interno entre empleados, no hay riesgo de que un cliente ajeno se meta en el medio.

9. Notificaciones al cliente (push desde la PWA + estado en tiempo real)

9.1 Canal principal: push desde la PWA

- El aviso de "tu pedido está listo" se dispara como **notificación push del navegador**, gestionada por el Service Worker de la PWA (en Angular, vía `@angular/service-worker / SwPush`) — no depende de WhatsApp ni de ningún proveedor externo de mensajería.

- El pedido de instalación + permiso de notificaciones se hace **justo después de confirmar el pago**, no al entrar a la app: es el momento de mayor motivación del cliente (ya pagó, quiere que le avisen) y se plantea como un paso liviano ("agregá esto a tu inicio para que te avisemos"), no como instalar una app de una store.
- La suscripción push queda atada al **pedido/sesión**, no a una cuenta de usuario (el cliente no tiene login) — se pierde si cambia de dispositivo, pero cubre el caso normal: un pedido, un celular, esa noche.

9.2 Limitación conocida: iOS sin instalar

- En Android/Chrome, el push funciona con solo otorgar el permiso, sin necesidad de instalar la PWA a pantalla de inicio.
- En iOS (Safari), el push **solo funciona si la PWA fue agregada a la pantalla de inicio**. Si el cliente no completa ese paso (o rechaza el permiso), no hay forma de que le llegue la notificación en background.

9.3 Red de respaldo: estado en tiempo real dentro de la app

- Para cubrir el caso de iOS sin instalar o permiso denegado, el cliente siempre tiene disponible la **pantalla de estado de su pedido**, que se actualiza **en tiempo real** (WebSocket o polling corto) sin necesitar refrescar manualmente.
- Estados visibles: en cola → en preparación → listo → entregado (los mismos de la sección 5).
- Esa misma pantalla es donde se muestra el **QR del cliente** para el retiro (ver sección 10).
- WhatsApp queda descartado como canal de aviso de "pedido listo" — evita depender de la API de WhatsApp Business (costo por mensaje, aprobación de plantillas, verificación de negocio) para algo que la propia PWA resuelve gratis.

10. Verificación con doble QR (retiro en barra)

Mismo identificador de pedido, mostrado en dos soportes distintos según el momento:

1. **QR del ticket de cocina** (impreso): lo escanea el bartender al terminar de preparar, para marcar "Listo".
2. **QR del cliente** (visible en la pantalla de estado del pedido dentro de la PWA): lo escanea quien entrega, al momento del retiro, para verificar que corresponde a ese pedido y marcarlo "Entregado".

No son dos códigos distintos — es el mismo QR/identificador del pedido, solo que aparece en dos lugares según la etapa.

Integración técnica del QR/lector

- Lectores de QR/código de barras USB/Bluetooth funcionan como **teclado (HID)**: no necesitan drivers ni librerías de cámara — el dispositivo "tipea" el contenido escaneado en un `<input>` enfocado y manda un `Enter`. Simplifica mucho la implementación.
- Comprar lector **2D** (imaging), que lee tanto QR como códigos de barra 1D.
- El ticket se imprime con QR usando comandos ESC/POS estándar en impresoras térmicas comunes.
- Mantener siempre un **input de búsqueda manual** como respaldo, por si falla el lector o se rompe/moja el ticket.

11. Pantalla / tablet del bartender

- **No es una pantalla por bartender** — es una pantalla/tablet **por estación de trabajo**, compartida por todos los que preparan ahí (mismo patrón que un KDS de cocina de restaurante). Reduce mucho la inversión en hardware.
- Muestra la cola de pedidos "Nuevos", ordenados por **antigüedad (FIFO)** desde el momento del pago — el objetivo del sistema es minimizar el tiempo total desde el pago hasta la entrega.
- El bartender **no está obligado al FIFO estricto**: puede elegir entre los **próximos 10 pedidos** de la cola, para poder agrupar pedidos del mismo trago y prepararlos juntos (más eficiente en tiempo total agregado, aunque rompa el orden estricto de un pedido individual).
- El ticket **no se imprime automáticamente** al confirmarse el pago — el bartender selecciona manualmente qué pedido(s) imprimir. Al imprimir, el pedido se saca de la cola de "Nuevos" para que otro bartender no lo tome también (funciona como un "tomar pedido" implícito).
- Al agrupar varios pedidos para prepararlos juntos, **se imprime un ticket individual por cada pedido** (no un ticket combinado) — cada uno mantiene su propio ticket y QR. El bartender simplemente los imprime y prepara juntos.

Qué muestra cada tarjeta de pedido en la tablet

- Número de orden
- Lista de ítems con cantidad y notas (ej. "sin hielo")
- Tipo de entrega: retiro en barra vs. mesa VIP (con número de mesa)
- Tiempo de espera desde el pago, con color según antigüedad (verde/amarillo/rojo) para priorización visual

12. Caja

- No arma pedidos manualmente (eso quedó descartado del diseño original).
- Rol 1: buscar el pedido por código cuando el cliente paga en efectivo, confirmar el cobro.

- Rol 2: coordinar la entrega de pedidos de mesa VIP que están "Listos" sin mozo asignado, dado que con el nuevo flujo tiene más disponibilidad que en el modelo actual con filas largas.

13. Consideraciones de costo / hardware (mantener bajo)

- 1 tablet compartida por estación de barra (no por bartender).
- 1 impresora térmica de tickets por barra.
- 1 lector de QR/código de barras por punto de entrega (no por persona).
- El mozo usa su propio celular, sin hardware dedicado.
- El cliente no necesita instalar nada para pedir y pagar — la PWA funciona igual desde el navegador. Solo se le sugiere instalarla **después de pagar**, para poder recibir la notificación push de "listo" (si no la instala, igual puede ver el estado en tiempo real dentro de la web).

14. Entidades propuestas (borrador — pendiente de formalizar en detalle)

Esto es un primer acercamiento a partir de todo lo hablado, todavía no se discutió el modelo de datos en profundidad.

- **Pedido:** id, estado, timestamp de pago, tipo de entrega (barra / mesa), mesa (si aplica), método de pago, total, QR/código identificador.
- **ItemPedido:** pedido, trago, cantidad, notas.
- **Trago** (menú): nombre, precio, disponibilidad/stock.
- **Mesa:** número/identificador, sector (VIP), cuenta asociada (si aplica).
- **CuentaVIP:** mesa asociada, saldo, código de acceso único.
- **Usuario/Rol:** cliente, cajero, bartender, mozo, admin — con login solo para roles internos (cajero/bartender/mozo/admin), el cliente no necesita cuenta.
- **SuscripciónPush:** pedido/sesión asociado, endpoint y claves de la Web Push API, plataforma/navegador — vive mientras dura el pedido, no es una entidad de usuario permanente.

15. Preguntas abiertas / a definir

- Qué pasa si un pedido en efectivo nunca se paga en caja: ¿timeout de cancelación automática, y a los cuántos minutos?
- Qué pasa con un pedido "Listo" que nadie retira por mucho tiempo (¿se re-prepara, se marca con alerta, se descarta?).

- Saldo insuficiente en cuenta VIP: bloquear pedido, cubrir diferencia con otro medio de pago, o permitir saldo negativo.
- Reparto de mesas entre mozos: zonas fijas vs. cola compartida (depende de cada boliche, no se resuelve a nivel de diseño general).
- Si el sistema es para un solo boliche o multi-tenant (varios boliches usando la misma plataforma) — decisión pendiente que impacta el modelo de datos y la necesidad de una pantalla de "elegir boliche/evento".
- Cuánto empuje darle al paso de "instalar la PWA" post-pago en iOS (¿opcional y discreto, o insistir con una pantalla dedicada?) — impacta cuánta gente termina dependiendo del fallback de estado en tiempo real en vez del push.
- Stack tecnológico: todavía no definido.
- Pantallas de administración (gestión de menú, mesas, usuarios, métricas): no se llegaron a diseñar en detalle todavía.

Flujo de Pedido

Diagrama de secuencia del flujo unificado

